

Core Java Programming Course Syllabus

Archer Infotech, Kothrud Pune | Classroom & live-online batches | Updated September 2026

Course Overview

This Core Java Programming course is designed to build a strong foundation in Java programming, object-oriented programming, exception handling, collections, file handling, multithreading, JDBC basics, functional programming, and modern Java features.

The course begins with programming fundamentals and Java syntax, then progresses through classes, objects, inheritance, polymorphism, interfaces, packages, generics, collections, streams, concurrency, memory management, debugging, testing, and practical project development.

The syllabus is suitable for beginners, students, engineering graduates, working professionals, and developers who want to build a strong Java foundation before moving to Advanced Java, Spring, Spring Boot, Microservices, Android, or enterprise application development.

Course Objectives

By the end of this course, learners will be able to:

- Understand Java architecture and the Java execution model.
- Write, compile, execute, and debug Java programs.
- Work with variables, data types, operators, conditions, loops, arrays, and strings.
- Apply object-oriented programming principles effectively.
- Design classes, interfaces, packages, and reusable components.
- Handle exceptions and build robust applications.
- Use Java Collections Framework and generics.
- Work with files and streams.
- Understand multithreading and concurrency basics.
- Use lambda expressions, functional interfaces, and Stream API.
- Understand Java memory management and garbage collection.
- Work with JDBC basics.
- Write unit tests for Java applications.
- Build console-based mini projects and capstone projects.
- Prepare for Java programming interviews and coding assessments.

Module 1: Introduction to Java

Topics

- Introduction to Computer Programming
- What is Java?
- History and Evolution of Java
- Features of Java
- Platform Independence
- Write Once, Run Anywhere
- Java Editions
- Java SE
- Jakarta EE Overview
- Java ME Overview
- Applications of Java
- Java vs C
- Java vs C++
- Java Release Model

- Long-Term Support Releases
- Introduction to Modern Java Versions
- Java Documentation and API Reference

Module 2: Java Architecture and Environment Setup

Topics

- Java Development Kit
- Java Runtime Environment
- Java Virtual Machine
- JDK vs JRE vs JVM
- Bytecode
- Java Compilation Process
- Class Loader Overview
- Just-In-Time Compilation
- Java HotSpot VM Overview
- Installing the JDK
- Configuring JAVA_HOME
- Configuring the PATH
- Checking Java Version
- Java Development Tools
- javac
- java
- javap
- javadoc
- jshell
- IDEs and Editors
- IntelliJ IDEA
- Eclipse
- Visual Studio Code
- Writing the First Java Program
- Compiling and Running from Command Line

Practical Exercises

- Hello World Program
- Compile and Run Using Terminal
- Run Java Using an IDE
- Explore Bytecode Using javap

Module 3: Structure of a Java Program

Topics

- Java Source File Structure
- Classes
- main() Method
- Statements
- Blocks
- Comments
- Single-Line Comments
- Multi-Line Comments
- Documentation Comments
- Identifiers
- Keywords

- Literals
- Naming Conventions
- Packages Overview
- Import Statements
- Semicolons
- Case Sensitivity
- Coding Style and Formatting

Module 4: Variables, Data Types, and Type Conversion

Topics

- Variables
- Variable Declaration
- Variable Initialization
- Local Variables
- Instance Variables
- Static Variables
- Constants
- `final` Keyword
- Primitive Data Types
- `byte`
- `short`
- `int`
- `long`
- `float`
- `double`
- `char`
- `boolean`
- Reference Data Types
- Default Values
- Numeric Literals
- Binary, Octal, Decimal, and Hexadecimal Literals
- Underscores in Numeric Literals
- Type Promotion
- Widening Conversion
- Narrowing Conversion
- Explicit Type Casting
- Overflow and Underflow
- Wrapper Classes Introduction

Practical Exercises

- Display Different Data Types
- Temperature Conversion
- Simple Interest
- Data Type Range Demonstration
- Type Casting Programs

Module 5: Operators and Expressions

Topics

- Arithmetic Operators
- Unary Operators
- Relational Operators

- Logical Operators
- Assignment Operators
- Compound Assignment Operators
- Increment and Decrement Operators
- Bitwise Operators
- AND
- OR
- XOR
- NOT
- Left Shift
- Right Shift
- Unsigned Right Shift
- Conditional / Ternary Operator
- instanceof Operator
- Operator Precedence
- Operator Associativity
- Expressions
- Short-Circuit Evaluation

Practical Exercises

- Even or Odd Number
- Largest of Two Numbers
- Largest of Three Numbers
- Leap Year
- Bitwise Operations

Module 6: Input and Output Basics

Topics

- Standard Input
- Standard Output
- `System.out.print()`
- `System.out.println()`
- `System.out.printf()`
- Formatted Output
- Scanner Class
- Reading Integer Values
- Reading Floating-Point Values
- Reading Characters
- Reading Strings
- `next()` vs `nextLine()`
- Input Buffer Issues
- Introduction to `BufferedReader`
- Input Validation Basics

Practical Exercises

- Student Details Input
- Arithmetic Calculator
- Formatted Mark Sheet
- Read Full Name and Address
- Numeric Input Validation

Module 7: Decision-Making Statements

Topics

- Control Flow
- if
- if-else
- Nested if
- else-if Ladder
- switch
- Traditional switch
- Modern Switch Expressions Overview
- case
- default
- break
- Conditional Operator
- Best Practices for Decision Logic

Practical Exercises

- Grade Calculator
- Positive, Negative, or Zero
- Vowel or Consonant
- Menu-Driven Calculator
- Electricity Bill Calculator

Module 8: Looping Statements**Topics**

- while
- do-while
- for
- Enhanced for
- Nested Loops
- Infinite Loops
- break
- continue
- Labeled break
- Labeled continue
- Loop Design Best Practices

Practical Exercises

- Multiplication Table
- Sum of Natural Numbers
- Factorial
- Fibonacci Series
- Reverse a Number
- Palindrome Number
- Armstrong Number
- Prime Number
- Prime Numbers in a Range
- Number Patterns
- Star Patterns

Module 9: Methods**Topics**

- Introduction to Methods
- Method Declaration
- Method Definition
- Method Invocation
- Parameters
- Arguments
- Return Types
- void Methods
- Method Overloading
- Variable-Length Arguments
- Pass-by-Value in Java
- Passing Object References
- Static Methods
- Instance Methods
- Recursive Methods
- Method Scope
- Local Variables
- Best Practices for Method Design

Practical Exercises

- Function-Based Calculator
- Factorial Using Recursion
- Fibonacci Using Recursion
- GCD and LCM
- Method Overloading Examples

Module 10: Arrays

Topics

- Introduction to Arrays
- One-Dimensional Arrays
- Array Declaration
- Array Creation
- Array Initialization
- Accessing Elements
- Traversing Arrays
- Array Length
- Passing Arrays to Methods
- Returning Arrays from Methods
- Two-Dimensional Arrays
- Multidimensional Arrays
- Jagged Arrays
- Anonymous Arrays
- Arrays Utility Class
- `java.util.Arrays`
- Sorting Arrays
- Searching Arrays
- Copying Arrays

Practical Exercises

- Sum and Average
- Maximum and Minimum
- Reverse an Array

- Linear Search
- Binary Search
- Bubble Sort
- Selection Sort
- Insertion Sort
- Matrix Addition
- Matrix Multiplication
- Matrix Transpose

Module 11: Strings

Topics

- Introduction to Strings
- String Objects
- String Literals
- String Pool
- String Immutability
- Creating Strings
- Comparing Strings
- == vs equals()
- Important String Methods
- length()
- charAt()
- substring()
- contains()
- indexOf()
- lastIndexOf()
- startsWith()
- endsWith()
- replace()
- trim()
- strip()
- toUpperCase()
- toLowerCase()
- split()
- join()
- String Concatenation
- String Conversion
- StringBuilder
- StringBuffer
- StringBuilder vs StringBuffer
- String Formatting
- Text Blocks Overview

Practical Exercises

- Reverse a String
- Check Palindrome
- Count Vowels and Consonants
- Character Frequency
- Word Count
- Remove Spaces
- Compare Strings
- StringBuilder Manipulation

Module 12: Object-Oriented Programming Fundamentals

Topics

- Procedural Programming vs Object-Oriented Programming
- OOP Principles
- Class
- Object
- Encapsulation
- Abstraction
- Inheritance
- Polymorphism
- Class Definition
- Object Creation
- Instance Variables
- Instance Methods
- Object State and Behavior
- Access Modifiers
- public
- private
- protected
- Default Access
- this Keyword
- Object References
- Anonymous Objects
- Class Members

Practical Exercises

- Student Class
- Employee Class
- Product Class
- Bank Account Class

Module 13: Constructors

Topics

- Introduction to Constructors
- Default Constructor
- No-Argument Constructor
- Parameterized Constructor
- Constructor Overloading
- Constructor Chaining
- this()
- super()
- Constructor Execution Order
- Private Constructors
- Object Initialization
- Constructor Best Practices

Practical Exercises

- Student Constructor Overloading
- Employee Object Initialization
- Constructor Chaining Example

Module 14: Encapsulation and Data Hiding

Topics

- What is Encapsulation?
- Data Hiding
- Private Fields
- Getter Methods
- Setter Methods
- Validation in Setters
- Immutable Classes
- Benefits of Encapsulation
- JavaBeans Naming Conventions Overview
- Defensive Copies Overview

Practical Exercises

- Encapsulated Bank Account
- Employee Salary Validation
- Immutable Student Object

Module 15: Inheritance

Topics

- Introduction to Inheritance
- extends Keyword
- Base Class
- Derived Class
- Single Inheritance
- Multilevel Inheritance
- Hierarchical Inheritance
- Multiple Inheritance Through Interfaces
- Constructors in Inheritance
- super Keyword
- Accessing Parent Members
- Method Overriding
- Inherited Members
- Protected Access
- IS-A Relationship
- HAS-A Relationship
- Composition vs Inheritance

Practical Exercises

- Person and Student
- Employee Hierarchy
- Vehicle Hierarchy
- Shape Hierarchy

Module 16: Polymorphism

Topics

- Introduction to Polymorphism
- Compile-Time Polymorphism
- Method Overloading
- Runtime Polymorphism

- Method Overriding
- Dynamic Method Dispatch
- Upcasting
- Downcasting
- instanceof
- Method Binding
- Covariant Return Types
- Object Class Methods
- Polymorphic Collections

Practical Exercises

- Shape Area Calculation
- Employee Salary Processing
- Payment Processing Hierarchy

Module 17: Abstraction

Topics

- What is Abstraction?
- Abstract Classes
- abstract Keyword
- Abstract Methods
- Concrete Methods in Abstract Classes
- Constructors in Abstract Classes
- Abstract Class Restrictions
- When to Use Abstract Classes
- Designing Base Abstractions

Practical Exercises

- Abstract Shape Class
- Abstract Employee Class
- Abstract Vehicle Class

Module 18: Interfaces

Topics

- Introduction to Interfaces
- Declaring Interfaces
- Implementing Interfaces
- Multiple Interface Implementation
- Interface Inheritance
- Constants in Interfaces
- Abstract Methods
- Default Methods
- Static Methods
- Private Interface Methods
- Functional Interfaces Introduction
- Marker Interfaces Overview
- Interface vs Abstract Class
- Loose Coupling
- Programming to Interfaces

Practical Exercises

- Payment Interface

- Printable Interface
- Multiple Interface Implementation
- Notification Service Interface

Module 19: Packages and Access Control

Topics

- Introduction to Packages
- Built-In Packages
- User-Defined Packages
- Creating Packages
- Importing Packages
- Static Imports
- Fully Qualified Class Names
- Package Naming Conventions
- Access Control Across Packages
- Classpath Basics
- Organizing Java Projects

Practical Exercises

- Create Utility Package
- Create Model and Service Packages
- Access Members Across Packages

Module 20: static, final, and Other Important Keywords

Topics

- static Variables
- static Methods
- Static Blocks
- Static Nested Classes
- final Variables
- final Methods
- final Classes
- Blank Final Variables
- this
- super
- instanceof
- native Overview
- transient Overview
- volatile Overview
- synchronized Overview

Module 21: Wrapper Classes and Autoboxing

Topics

- Primitive Types vs Wrapper Classes
- Wrapper Classes
- Byte
- Short
- Integer
- Long
- Float

- Double
- Character
- Boolean
- Boxing
- Unboxing
- Autoboxing
- Auto-Unboxing
- Parsing Strings
- Converting Numbers to Strings
- Integer Cache Overview
- Wrapper Utility Methods

Practical Exercises

- String to Integer Conversion
- Integer to String Conversion
- Wrapper Comparison Examples

Module 22: Enums

Topics

- Introduction to Enums
- Declaring Enums
- Enum Constants
- Enum Methods
- `values()`
- `valueOf()`
- Enum Constructors
- Fields in Enums
- Methods in Enums
- Enum in Switch
- Enum Best Practices

Practical Exercises

- Days of Week
- Order Status
- User Roles
- Payment Status

Module 23: Nested and Inner Classes

Topics

- Nested Classes
- Static Nested Classes
- Member Inner Classes
- Local Inner Classes
- Anonymous Inner Classes
- Accessing Outer Class Members
- Use Cases
- Inner Classes vs Lambdas

Module 24: Exception Handling

Topics

- Introduction to Exceptions
- Error vs Exception
- Exception Hierarchy
- Checked Exceptions
- Unchecked Exceptions
- try
- catch
- Multiple Catch Blocks
- Multi-Catch
- finally
- throw
- throws
- Custom Exceptions
- Exception Propagation
- Try-with-Resources
- Suppressed Exceptions Overview
- Best Practices for Exception Handling

Practical Exercises

- Division-by-Zero Handling
- Invalid Age Exception
- Invalid Balance Exception
- File Exception Handling
- Custom Business Exception

Module 25: Java Collections Framework

Topics

- Introduction to Collections
- Collection Hierarchy
- Collection Interface
- List
- Set
- Queue
- Deque
- Map
- Collections vs Arrays
- Generics with Collections
- Iteration Techniques
- Iterator
- ListIterator
- Enhanced for
- forEach

Module 26: List Implementations

Topics

- ArrayList
- LinkedList
- Vector
- Stack
- List Operations
- Adding Elements

- Removing Elements
- Updating Elements
- Searching
- Sorting
- Iterating
- ArrayList vs LinkedList
- Vector vs ArrayList

Practical Exercises

- Student List
- Employee List
- To-Do List
- Sorting Custom Objects

Module 27: Set Implementations

Topics

- HashSet
- LinkedHashSet
- TreeSet
- Unique Elements
- Hashing Basics
- Sorted Sets
- Set Operations
- Union
- Intersection
- Difference
- Custom Objects in Sets
- `equals()` and `hashCode()`

Practical Exercises

- Remove Duplicate Values
- Unique Student Records
- Sorted Unique Values

Module 28: Map Implementations

Topics

- Map Interface
- Key-Value Pairs
- HashMap
- LinkedHashMap
- TreeMap
- Hashtable
- Adding and Updating Entries
- Removing Entries
- Iterating Maps
- Entry Set
- Key Set
- Values Collection
- Null Keys and Values
- Map Utility Methods
- `compute`

- merge
- putIfAbsent
- Custom Keys
- equals() and hashCode()

Practical Exercises

- Word Frequency Counter
- Student Marks Map
- Product Price Lookup
- Employee Directory

Module 29: Queue and Deque

Topics

- Queue Interface
- PriorityQueue
- ArrayDeque
- FIFO
- LIFO
- Queue Operations
- offer
- poll
- peek
- Priority-Based Processing
- Deque Operations

Practical Exercises

- Task Queue
- Priority-Based Job Processing
- Browser History Simulation

Module 30: Generics

Topics

- Introduction to Generics
- Generic Classes
- Generic Methods
- Generic Interfaces
- Type Parameters
- Multiple Type Parameters
- Bounded Type Parameters
- Wildcards
- Unbounded Wildcards
- Upper-Bounded Wildcards
- Lower-Bounded Wildcards
- Type Erasure Overview
- Generic Restrictions
- Benefits of Generics

Practical Exercises

- Generic Box
- Generic Pair
- Generic Maximum Method
- Generic Repository

Module 31: Comparable and Comparator

Topics

- Natural Ordering
- Comparable
- compareTo()
- Custom Ordering
- Comparator
- compare()
- Chained Comparators
- Comparator.comparing()
- Reversed Ordering
- Sorting Collections
- Comparable vs Comparator

Practical Exercises

- Sort Students by Marks
- Sort Employees by Salary
- Multi-Level Sorting

Module 32: Object Class

Topics

- java.lang.Object
- toString()
- equals()
- hashCode()
- getClass()
- clone() Overview
- finalize() Historical Overview and Deprecation Context
- Object Equality
- Reference Equality
- Contract Between equals() and hashCode()
- Best Practices

Module 33: Java I/O and File Handling

Topics

- Introduction to I/O
- Byte Streams
- Character Streams
- InputStream
- OutputStream
- Reader
- Writer
- FileInputStream
- FileOutputStream
- FileReader
- FileWriter
- BufferedInputStream
- BufferedOutputStream
- BufferedReader
- BufferedWriter

- PrintWriter
- File Class
- File and Directory Operations
- Try-with-Resources
- Reading Text Files
- Writing Text Files
- Copying Files

Practical Exercises

- Write Data to File
- Read Data from File
- Copy File
- Count Words and Lines
- Student Record File

Module 34: Serialization**Topics**

- Introduction to Object Serialization
- Serializable
- ObjectOutputStream
- ObjectInputStream
- Serialization Process
- Deserialization
- serialVersionUID
- transient
- Static Fields and Serialization
- Serialization Best Practices
- Security Considerations

Practical Exercises

- Serialize Student Object
- Deserialize Employee Object
- Store Collection of Objects

Module 35: NIO and NIO.2 Basics**Topics**

- Introduction to NIO
- Path
- Paths
- Files
- Reading Files with Files
- Writing Files with Files
- Creating Directories
- Copying Files
- Moving Files
- Deleting Files
- File Attributes Overview
- Directory Traversal
- Buffered I/O with NIO

Module 36: Date and Time API

Topics

- Legacy Date APIs Overview
- `java.time`
- `LocalDate`
- `LocalTime`
- `LocalDateTime`
- `ZonedDateTime`
- `Instant`
- `Period`
- `Duration`
- `DateTimeFormatter`
- Parsing Dates
- Formatting Dates
- Date Arithmetic
- Time Zones Overview

Practical Exercises

- Calculate Age
- Find Difference Between Dates
- Format Date and Time
- Appointment Scheduler

Module 37: Regular Expressions**Topics**

- Introduction to Regular Expressions
- Pattern
- Matcher
- Character Classes
- Quantifiers
- Groups
- Anchors
- Validation Patterns
- Search and Replace
- Splitting Strings

Practical Exercises

- Email Validation
- Mobile Number Validation
- Password Validation
- Extract Numbers from Text

Module 38: Lambda Expressions**Topics**

- Introduction to Functional Programming
- Lambda Syntax
- Parameters
- Return Values
- Lambda with One Parameter
- Lambda with Multiple Parameters
- Block Lambdas
- Variable Capture

- Effectively Final Variables
- Lambdas vs Anonymous Classes
- Lambdas with Collections

Practical Exercises

- Sort Using Lambda
- Filter a Collection
- Custom Runnable Using Lambda

Module 39: Functional Interfaces

Topics

- What is a Functional Interface?
- @FunctionalInterface
- Built-In Functional Interfaces
- Predicate
- Function
- Consumer
- Supplier
- UnaryOperator
- BinaryOperator
- Primitive Specializations Overview
- Method References
- Static Method Reference
- Instance Method Reference
- Constructor Reference

Practical Exercises

- Predicate-Based Filtering
- Function Transformations
- Consumer Processing
- Supplier Examples

Module 40: Stream API

Topics

- Introduction to Streams
- Stream vs Collection
- Creating Streams
- Intermediate Operations
- filter
- map
- flatMap
- distinct
- sorted
- limit
- skip
- peek
- Terminal Operations
- forEach
- collect
- reduce
- count

- min
- max
- findFirst
- findAny
- anyMatch
- allMatch
- noneMatch
- Collectors
- Grouping
- Partitioning
- Joining
- Parallel Streams Overview
- Stream Best Practices

Practical Exercises

- Filter Employees by Salary
- Sort Students by Marks
- Group Products by Category
- Calculate Totals with Reduce
- Find Duplicate Values

Module 41: Optional

Topics

- Introduction to Optional
- Creating Optional Objects
- of
- ofNullable
- empty
- isPresent
- ifPresent
- orElse
- orElseGet
- orElseThrow
- map
- flatMap
- Avoiding NullPointerException
- Optional Best Practices

Module 42: Multithreading Fundamentals

Topics

- Process vs Thread
- Advantages of Multithreading
- Thread Lifecycle
- Creating Threads
- Extending Thread
- Implementing Runnable
- Starting Threads
- sleep()
- join()
- Thread Priorities
- Daemon Threads

- Interrupting Threads
- Thread States
- Race Conditions
- Thread Safety Basics

Practical Exercises

- Create Multiple Threads
- Parallel Task Execution
- Thread Lifecycle Demonstration

Module 43: Synchronization and Concurrency Basics

Topics

- Shared Resources
- Race Conditions
- Critical Sections
- synchronized
- Synchronized Methods
- Synchronized Blocks
- Intrinsic Locks
- Deadlocks
- Livelocks Overview
- Starvation Overview
- wait()
- notify()
- notifyAll()
- Producer-Consumer Problem
- volatile
- Atomicity Basics

Module 44: Java Concurrency Utilities

Topics

- java.util.concurrent
- Executor Framework
- Executor
- ExecutorService
- Thread Pools
- Callable
- Future
- Scheduled Executors
- CompletableFuture Introduction
- Concurrent Collections Overview
- ConcurrentHashMap
- Locks Overview
- ReentrantLock
- Atomic Classes Overview

Practical Exercises

- Fixed Thread Pool
- Callable and Future
- Asynchronous Task with CompletableFuture

Module 45: Memory Management and Garbage Collection

Topics

- JVM Memory Areas
- Heap
- Stack
- Metaspace
- Method Area Concept
- Object Allocation
- References
- Object Eligibility for Garbage Collection
- Garbage Collector Basics
- Automatic Memory Management
- Strong References
- Weak References Overview
- Soft References Overview
- Phantom References Overview
- Memory Leaks in Java
- `System.gc()` Considerations
- Finalization Deprecation Context
- Basic JVM Monitoring Concepts

Module 46: Java Annotations

Topics

- Introduction to Annotations
- Built-In Annotations
- `@Override`
- `@Deprecated`
- `@SuppressWarnings`
- `@FunctionalInterface`
- Custom Annotations Overview
- Annotation Targets
- Retention Policies
- Runtime Annotation Basics
- Reflection with Annotations Overview

Module 47: Reflection API Basics

Topics

- Introduction to Reflection
- `Class` Object
- Getting Class Information
- Inspecting Fields
- Inspecting Methods
- Inspecting Constructors
- Creating Objects Dynamically Overview
- Access Control Considerations
- Reflection Use Cases
- Performance and Security Considerations

Module 48: Records, Sealed Classes, and Modern Java Features

Topics

- Local Variable Type Inference with var
- Switch Expressions
- Text Blocks
- Pattern Matching Overview
- Records
- Record Components
- Records vs Traditional Classes
- Sealed Classes
- sealed
- permits
- non-sealed
- Pattern Matching for instanceof
- Modern Switch Pattern Matching Overview
- Immutable Data Modeling
- Overview of Recent Java Language Enhancements

Module 49: JDBC Fundamentals

Topics

- Introduction to JDBC
- JDBC Architecture
- JDBC Drivers
- Database Connection
- DriverManager
- Connection
- Statement
- PreparedStatement
- ResultSet
- Executing Queries
- Executing Updates
- Parameterized Queries
- SQL Injection Prevention
- Closing Database Resources
- Try-with-Resources
- Transactions Introduction
- Commit and Rollback Overview

Practical Exercises

- Connect Java to Database
- Insert Student Record
- Retrieve Records
- Update Employee Record
- Delete Product Record
- CRUD Console Application

Module 50: Debugging and Error Diagnosis

Topics

- Syntax Errors
- Compilation Errors
- Runtime Errors
- Logical Errors

- Reading Stack Traces
- IDE Debugging
- Breakpoints
- Step Into
- Step Over
- Step Out
- Watch Expressions
- Inspecting Variables
- Logging Basics
- Assertions
- Debugging Exceptions
- Debugging Multithreaded Programs Overview

Module 51: Unit Testing Basics

Topics

- Importance of Testing
- Unit Testing Concepts
- Test Cases
- Positive Testing
- Negative Testing
- Boundary Testing
- Regression Testing
- Introduction to JUnit
- Test Methods
- Assertions
- Setup and Teardown
- Testing Exceptions
- Parameterized Tests Overview
- Test-Driven Development Introduction

Module 52: Build Tools Introduction

Topics

- Why Build Tools?
- Dependency Management
- Project Builds
- Maven Overview
- Maven Project Structure
- pom.xml
- Dependencies
- Build Lifecycle
- Gradle Overview
- Maven vs Gradle
- Creating a Simple Maven Java Project

Module 53: Code Quality and Professional Java Practices

Topics

- Java Naming Conventions
- Clean Code Principles
- Method Design
- Class Design

- Avoiding Duplicate Code
- Immutability
- Defensive Programming
- Proper Exception Handling
- Logging Instead of Printing
- Avoiding Null-Related Issues
- Choosing Correct Collections
- Resource Management
- Try-with-Resources
- Thread Safety Awareness
- Secure Coding Basics
- Code Reviews
- Java Documentation
- Javadoc
- Static Analysis Overview
- Checkstyle / SpotBugs Overview

Module 54: Problem-Solving and Algorithmic Thinking

Topics

- Understanding Problem Statements
- Identifying Inputs and Outputs
- Constraints
- Algorithms
- Pseudocode
- Dry Runs
- Edge Cases
- Time Complexity Introduction
- Space Complexity Introduction
- Big-O Notation Basics
- Choosing Appropriate Data Structures
- Recursion
- Searching
- Sorting
- Common Problem-Solving Patterns

Practice Categories

- Number Problems
- Pattern Problems
- Array Problems
- String Problems
- Collection Problems
- OOP Problems
- Recursion Problems
- Exception Handling Problems
- Stream API Problems

Module 55: Data Structures with Core Java

Topics

- Arrays
- Dynamic Collections
- Linked List Concepts

- Stack
- Queue
- Priority Queue
- Hashing
- Sets
- Maps
- Trees Overview
- Binary Search Tree Introduction
- Searching Algorithms
- Sorting Algorithms

Algorithms

- Linear Search
- Binary Search
- Bubble Sort
- Selection Sort
- Insertion Sort
- Merge Sort Overview
- Quick Sort Overview

Module 56: Git and GitHub for Java Developers

Topics

- Introduction to Version Control
- Installing Git
- Creating a Repository
- `git init`
- `git add`
- `git commit`
- `git status`
- `git log`
- Branching Basics
- Merging
- GitHub Repository Creation
- Push and Pull
- Pull Requests Overview
- README Documentation
- `.gitignore`
- Java-Specific `.gitignore`
- Building a Programming Portfolio

Module 57: AI-Assisted Java Development

Topics

- Using AI Coding Assistants Responsibly
- Generating Boilerplate Code
- Explaining Java Code
- Understanding Compiler Errors
- Debugging with AI Assistance
- Generating Unit Tests
- Refactoring Suggestions
- Code Review with AI
- Creating Documentation

- Generating Example Data
- Reviewing Stream API Code
- Reviewing Exception Handling
- Security Risks of Blindly Using Generated Code
- Validating AI-Generated Solutions
- Prompting Techniques for Java Programming

Recommended Practice

Students should learn to:

- Ask AI for explanations rather than only solutions.
- Verify generated code manually.
- Run tests and validate edge cases.
- Review exception handling.
- Check thread-safety assumptions.
- Analyze performance of generated code.
- Understand every generated line before submission.

Module 58: Mini Projects

Students should complete multiple mini projects such as:

- Calculator Application
- Student Grade Management System
- Employee Management System
- Library Management System
- Banking Management System
- Inventory Management System
- Contact Management System
- Quiz Application
- Expense Tracker
- Task Management Application
- Hotel Reservation System
- Vehicle Rental System
- File-Based Address Book
- JDBC-Based Student Management System
- Employee Payroll System
- Console-Based E-Commerce Cart
- Banking Application with Exception Handling
- Collection-Based Library System

Module 59: Capstone Project

Learners should build one complete Java application using the concepts covered in the course.

Suggested Capstone Projects

- Library Management System
- Hospital Management System
- Student Information System
- Banking Management System
- Employee Payroll System
- Inventory and Billing System
- Hotel Reservation System
- Vehicle Rental System
- Task Management System

- JDBC-Based CRUD Application

Capstone Requirements

The project should demonstrate:

- Object-oriented design
- Classes and objects
- Encapsulation
- Inheritance where appropriate
- Polymorphism
- Interfaces
- Packages
- Exception handling
- Collections
- Generics
- File handling or JDBC
- Lambda expressions
- Stream API
- Input validation
- Modular code organization
- Unit testing basics
- Git-based version control
- README documentation

Module 60: Core Java Interview Preparation

Core Java Interview Topics

- JDK, JRE, and JVM
- Java Compilation Process
- Primitive and Reference Types
- Stack vs Heap
- Strings and String Pool
- String vs StringBuilder vs StringBuffer
- `==` vs `equals()`
- `equals()` and `hashCode()`
- Classes and Objects
- Constructors
- `this` and `super`
- Inheritance
- Polymorphism
- Abstraction
- Interfaces
- Abstract Class vs Interface
- Method Overloading vs Overriding
- Access Modifiers
- `static`
- `final`
- Exception Handling
- Checked vs Unchecked Exceptions
- Collections Framework
- ArrayList vs LinkedList
- HashSet vs TreeSet
- HashMap vs Hashtable
- Comparable vs Comparator

- Generics
- Wrapper Classes
- Autoboxing
- Serialization
- Multithreading
- Synchronization
- Executor Framework
- Lambda Expressions
- Functional Interfaces
- Stream API
- Optional
- Garbage Collection
- JDBC Basics
- Modern Java Features

Interview Activities

- Conceptual Questions
- Output-Based Questions
- Debugging Questions
- Coding Challenges
- Collection-Based Problems
- OOP Design Questions
- Stream API Problems
- Mock Interviews
- Timed Coding Tests

Recommended Lab Assignments

Learners should complete at least:

- 20 Basic Syntax and Operator Programs
- 20 Conditional and Looping Programs
- 15 Method and Recursion Programs
- 20 Array and String Programs
- 20 OOP Programs
- 10 Inheritance and Polymorphism Programs
- 10 Interface and Abstraction Programs
- 15 Exception Handling Programs
- 20 Collections and Generics Exercises
- 15 Lambda and Stream API Exercises
- 10 File Handling Programs
- 5 Multithreading Exercises
- 5 JDBC Exercises
- 5 Mini Projects
- 1 Capstone Project

Suggested Course Duration

- Section — Suggested Duration
- Java Fundamentals — 12–15 Hours
- Control Flow, Methods, Arrays, and Strings — 18–22 Hours
- Object-Oriented Programming — 20–25 Hours
- Exceptions, Packages, and Core APIs — 12–15 Hours
- Collections and Generics — 18–22 Hours
- I/O, Serialization, and NIO — 10–12 Hours

- Lambda, Functional Interfaces, and Streams — 12–15 Hours
- Multithreading and Concurrency — 12–15 Hours
- JDBC, Testing, and Build Tools — 10–12 Hours
- Projects and Interview Preparation — 20–25 Hours

Recommended Total Duration: approximately **145–180 hours**, depending on practical depth, assignments, and project work.

Prerequisites

No prior programming experience is required.

Basic familiarity with computers, files, folders, and command-line usage is helpful.

Learners who already know C or C++ may progress faster through the introductory programming modules.

Tools and Technologies

- Java
- JDK
- JVM
- IntelliJ IDEA
- Eclipse
- Visual Studio Code
- JShell
- Git
- GitHub
- Maven
- Gradle Overview
- JUnit
- MySQL / PostgreSQL for JDBC Practice
- Command Line / Terminal

Learning Outcomes

After completing the course, learners should be able to:

- Write structured and object-oriented Java applications.
- Understand Java execution, memory, and JVM fundamentals.
- Apply encapsulation, inheritance, polymorphism, and abstraction.
- Use interfaces and packages to design modular applications.
- Handle errors using Java exception handling.
- Work effectively with collections and generics.
- Process data using lambda expressions and Stream API.
- Perform file operations and object serialization.
- Build basic multithreaded applications.
- Connect Java applications to relational databases using JDBC.
- Debug and test Java applications.
- Use Git and GitHub professionally.
- Build portfolio-ready Java projects.
- Solve Core Java interview and coding problems.
- Progress confidently to enterprise Java development.

Recommended Progression After Core Java

After completing Core Java, learners can continue with:

- Advanced Java
- JDBC in Depth
- Servlets
- JSP
- Maven and Gradle
- Spring Framework
- Spring Boot
- Spring Data JPA
- Hibernate
- REST API Development
- Microservices
- Spring Security
- Spring Cloud
- Docker and Kubernetes
- Cloud Deployment
- Spring AI
- Data Structures and Algorithms in Java
- System Design for Java Developers

Enquire or book a free demo session. Archer Infotech, Flat No. 12, Divyadarshan Housing Society, behind Kothrud Bus Stand Road, Kothrud, Pune 411038. Phone +91 9850 678451. Email info@archerinfotech.in. Web archerinfotech.in.